

# Trabajo Práctico 5 – Sistemas de Computación



## Integrantes:

- Brigido Tomás
- Siñuka Javier
- Tosolini Agustín

Link del repositorio: [https://github.com/Javier-Sinuka/computer\\_systems\\_ramnt/tree/main/TP5](https://github.com/Javier-Sinuka/computer_systems_ramnt/tree/main/TP5)

|          |                                            |           |
|----------|--------------------------------------------|-----------|
| <b>1</b> | <b>Desarrollo</b>                          | <b>3</b>  |
| 1.1      | Arquitectura general del sistema           | 3         |
| 1.2      | Generación de señales con Arduino          | 4         |
| 1.3      | Preparación para compilación cruzada       | 5         |
| 1.4      | Implementación del módulo de kernel        | 7         |
| 1.5      | Compilación del módulo en la notebook      | 8         |
| 1.6      | Transferencia y acceso por SSH             | 9         |
| 1.7      | Carga y verificación del módulo            | 9         |
| 1.8      | Lector C de espacio de usuario             | 10        |
| 1.9      | Envío de muestras hacia la notebook        | 11        |
| 1.10     | Dashboard web en Docker                    | 12        |
| <b>2</b> | <b>Análisis técnico</b>                    | <b>14</b> |
| 2.1      | Separación entre kernel space y user space | 14        |
| 2.2      | Coherencia de los datos observados         | 14        |
| 2.3      | Consideraciones de hardware                | 14        |
| <b>3</b> | <b>Conclusión</b>                          | <b>16</b> |

# 1 Desarrollo

En este trabajo se implementó un flujo completo de adquisición, transporte y visualización de señales digitales utilizando una Raspberry Pi 5, un módulo cargable del kernel, un programa de espacio de usuario y una aplicación web dockerizada ejecutada en la notebook. La arquitectura resultante vincula dos dominios de ejecución claramente diferenciados: por un lado, el espacio del kernel, responsable de acceder a los GPIO y exponerlos mediante un dispositivo de carácter; por otro lado, el espacio de usuario, encargado de leer ese dispositivo, serializar las muestras y transportarlas hacia una interfaz gráfica.

El procedimiento se organizó en una secuencia práctica: primero se generó una señal binaria conocida desde Arduino, luego se construyó y cargó el driver en la Raspberry Pi, posteriormente se compiló un lector C para ARM64 y, finalmente, se enviaron las mediciones por SSH hacia una instancia Docker local para su representación.

## 1.1 Arquitectura general del sistema

El sistema desarrollado puede resumirse como una cadena de adquisición de dos bits. El sketch de Arduino produce las combinaciones 00, 01, 10 y 11; esas señales se conectan al circuito de entrada de la Raspberry Pi; el módulo de kernel lee los GPIO BCM configurados; el binario `gpio_jsonl_reader-aarch64` convierte cada lectura en una línea JSON; y la notebook recibe esas líneas para enviarlas al dashboard disponible en <http://localhost:8080>.

Desde el punto de vista del contrato entre componentes, el punto central es el archivo de dispositivo `/dev/tp5_gpio`. Este archivo no es un archivo regular: representa una interfaz de carácter registrada por el módulo de kernel. Cada lectura sobre él dispara una nueva medición de las entradas GPIO y devuelve un texto con el siguiente formato:

```
1  gpio_a=17 value=0
2  gpio_b=27 value=1
```

Sobre esa base, el programa de usuario puede operar sin conocer los detalles internos del driver, limitándose a abrir, leer y cerrar el dispositivo en ciclos sucesivos.

## 1.2 Generación de señales con Arduino

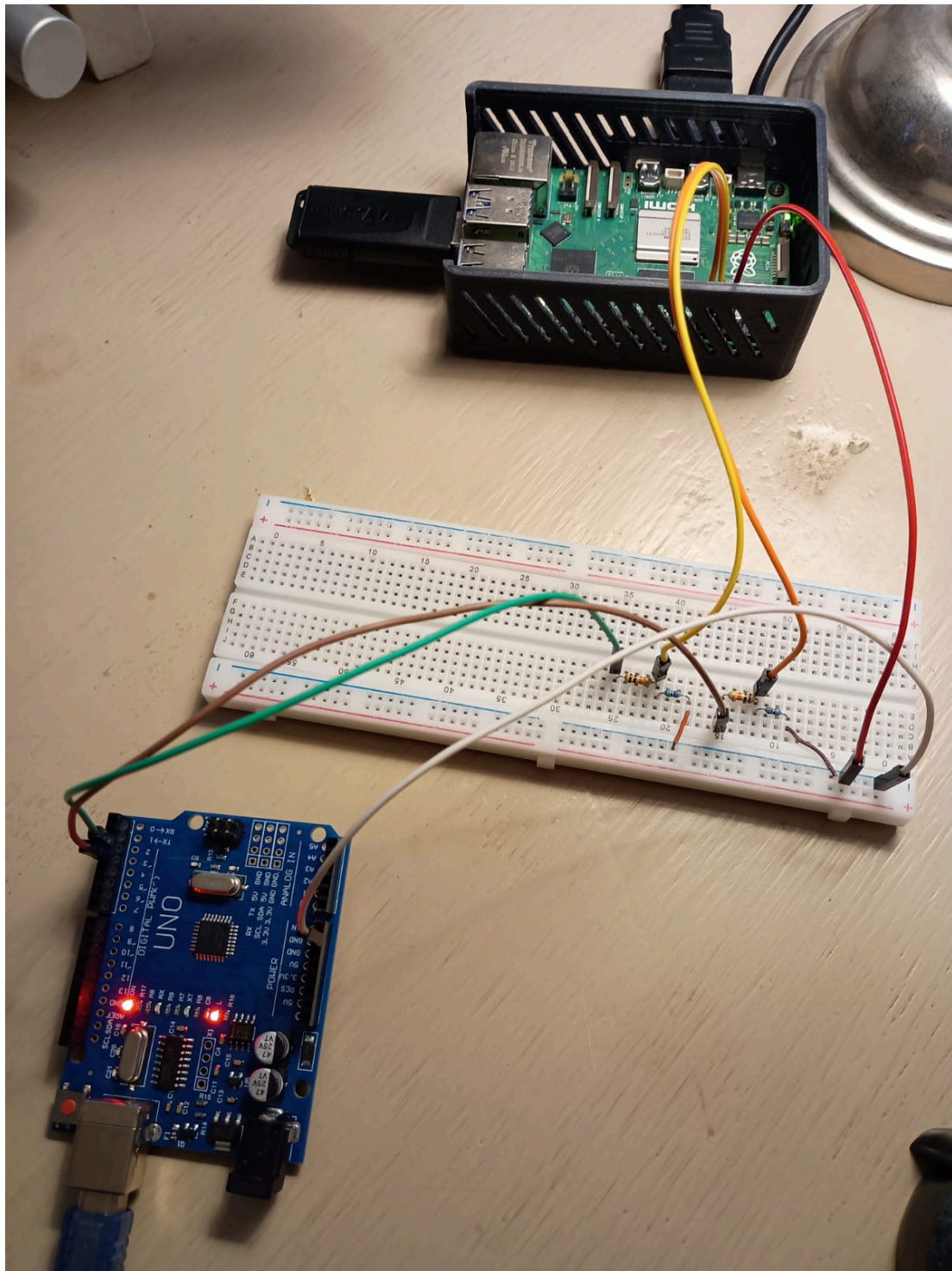
Para disponer de una entrada controlada y repetible, se utilizó el sketch `tools/arduino/waveform_former.ino`. El programa configura los pines digitales 2 y 3 como salidas y los conmuta con una espera de 500 ms entre estados. La secuencia generada es:

00 -> 01 -> 10 -> 11

De este modo, las entradas de la Raspberry Pi reciben una forma de onda digital conocida. Esto facilita verificar si el módulo de kernel, el lector C y el dashboard conservan correctamente la información binaria a través de todo el flujo.

```
1  const int gpioA = 2;
2  const int gpioB = 3;
```

En la implementación física se conectaron la Raspberry Pi, el circuito sobre protoboard y la placa Arduino utilizada para generar la señal. Dado que las entradas GPIO de la Raspberry Pi 5 trabajan a 3.3 V y no son tolerantes a 5 V, el circuito debe incluir adaptación de nivel cuando la placa generadora entrega niveles lógicos de 5 V.



1.1: Circuito físico utilizado para generar y leer las señales digitales

## 1.3 Preparación para compilación cruzada

La compilación del módulo no se realizó directamente en la Raspberry Pi, sino en la notebook. Esta decisión reduce el uso de recursos de la placa, evita trabajo intensivo sobre la tarjeta SD y permite aprovechar el entorno de desarrollo local. Como la Raspberry Pi 5

ejecuta un sistema ARM64/AArch64, se preparó una toolchain de compilación cruzada basada en `aarch64-linux-gnu-`.

La compilación cruzada fue una etapa central del TP porque el equipo de desarrollo y el equipo de ejecución no compartían la misma arquitectura. La notebook actuó como máquina host, encargada de compilar, mientras que la Raspberry Pi 5 actuó como máquina target, encargada de ejecutar el módulo y el lector. Por ese motivo, no alcanzaba con invocar un `make` convencional: era necesario indicarle explícitamente al sistema de build que el resultado debía ser compatible con ARM64 y con el kernel concreto instalado en la Raspberry Pi.

De acuerdo con el procedimiento documentado en `documentation/cross-compilation.txt`, la notebook debe contar con herramientas de construcción, dependencias del kernel y toolchain ARM64:

```
1  sudo apt update
2  sudo apt install build-essential bc bison flex libssl-dev libelf-dev libc6-
```

Además, para compilar un módulo externo es necesario disponer del árbol de fuentes del kernel correspondiente a la versión ejecutada por la Raspberry Pi. La versión debe relevarse en la placa mediante:

```
uname -r
```

Esta verificación es importante porque un módulo de kernel no se vincula contra una ABI genérica independiente del sistema. El archivo `.ko` queda asociado a la versión y configuración del kernel para el cual fue construido. Si el árbol de kernel preparado en la notebook no coincide con el kernel que ejecuta la Raspberry Pi, pueden aparecer errores de carga, incompatibilidades de símbolos o diferencias en la cadena `vermagic`.

Luego, en la notebook, se prepara el entorno apuntando a ARM64:

```
1  export ARCH=arm64
2  export CROSS_COMPILE=aarch64-linux-gnu-
3  make bcm2712_defconfig
```

cruzada que se deben utilizar. La regla `bcm2712_defconfig` selecciona una configuración base adecuada para Raspberry Pi 5, y `modules_prepare` genera los archivos auxiliares, cabeceras preparadas y scripts necesarios para compilar módulos externos sin construir todo el kernel completo.

El uso de `bcm2712_defconfig` corresponde a Raspberry Pi 5. También es relevante que esta placa delega parte de la gestión de GPIO en el chip RP1; por ello, el módulo no accede a registros físicos de GPIO mediante direcciones fijas, sino que utiliza la API del kernel para GPIO. Esta decisión evita acoplar el driver a mapas de memoria particulares y permite que la lectura de entradas digitales se realice mediante las abstracciones provistas por el kernel.

## 1.4 Implementación del módulo de kernel

El driver desarrollado se encuentra en `tools/gpio_module/dual_gpio_input_module.c`. Su propósito es leer dos señales GPIO de entrada y exponerlas a espacio de usuario mediante el dispositivo `/dev/tp5_gpio`. Para ello, el módulo define tres parámetros principales:

```
1  static int gpio_a = 17;
2  static int gpio_b = 27;
3  static char *gpio_chip_label = "pinctrl-rp1";
```

Los GPIO 17 y 27 son los valores por defecto utilizados por el módulo, mientras que `pinctrl-rp1` identifica el controlador de GPIO correspondiente a la Raspberry Pi 5. Al cargarse, el driver valida que ambos GPIO sean distintos, obtiene los descriptores mediante la infraestructura de GPIO del kernel y los configura como entradas.

El módulo registra un dispositivo de carácter mediante `alloc_chrdev_region`, `cdev_add`, `class_create` y `device_create`. La operación de lectura queda asociada a la función `dual_gpio_read`, que toma los valores actuales con `gpiod_get_value` y los copia hacia espacio de usuario mediante `simple_read_from_buffer`.

La salida entregada por el driver tiene el siguiente formato textual:

```
1  gpio_a=17 value=0
2  gpio_b=27 value=1
```

conserva la responsabilidad de interactuar con el hardware, mientras que el programa de usuario procesa una interfaz estable basada en texto.

## 1.5 Compilación del módulo en la notebook

El `Makefile` de `tools/gpio_module` compila `dual_gpio_input_module.c` como módulo externo:

```
obj-m := dual_gpio_input_module.o
```

La compilación se realiza apuntando al árbol de kernel preparado para Raspberry Pi y a la arquitectura ARM64. Un comando representativo es:

```
1  cd tools/gpio_module
2  make KERNEL_DIR=/ruta/al/kernel/raspberrypi-linux \
3      ARCH=arm64 \
4      CROSS_COMPILE=aarch64-linux-gnu- \
5      CC=aarch64-linux-gnu-gcc-14
```

En este comando, `KERNEL_DIR` referencia el árbol de kernel preparado en la notebook; `M=$(CURDIR)`, definido dentro del `Makefile`, le indica al sistema de construcción del kernel que compile el módulo externo ubicado en la carpeta actual; `ARCH` y `CROSS_COMPILE` mantienen la arquitectura de destino; y `CC` fuerza el compilador ARM64 disponible en el entorno. De esta forma, aunque el comando se ejecuta físicamente en la notebook, el resultado generado está preparado para cargarse en la Raspberry Pi.

Como resultado se obtiene el archivo:

```
dual_gpio_input_module.ko
```

Este archivo es el objeto cargable por el kernel de la Raspberry Pi. Antes de transferirlo, puede verificarse su arquitectura con:

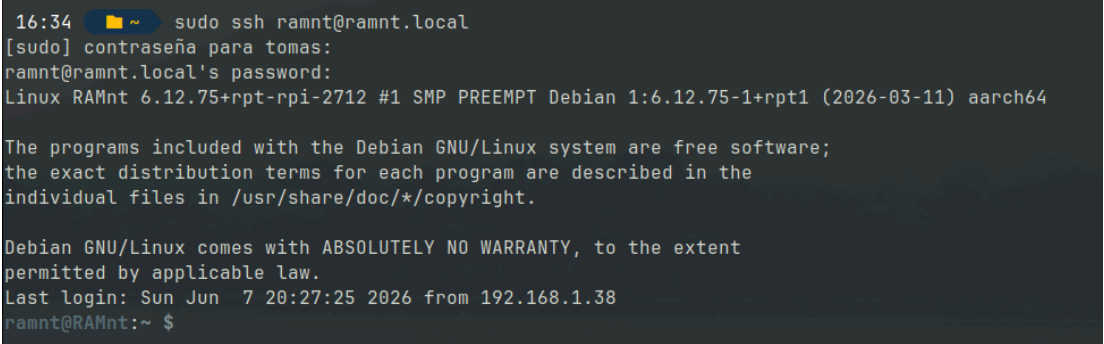


para la arquitectura correcta y que el trabajo pesado de compilación quedó efectivamente resuelto fuera de la Raspberry Pi.

## 1.6 Transferencia y acceso por SSH

Una vez compilado el módulo, se lo transfirió a la Raspberry Pi mediante `scp` y se trabajó sobre la placa a través de SSH. En la evidencia de conexión se observa el acceso al host `ramnt.local` con el usuario `ramnt`, ejecutando un sistema Debian GNU/Linux sobre arquitectura `aarch64`.

```
1  scp -o PubkeyAuthentication=no dual_gpio_input_module.ko ramnt@ramnt.local:
2  ssh -o PubkeyAuthentication=no ramnt@ramnt.local
```



```
16:34 ~ sudo ssh ramnt@ramnt.local
[sudo] contraseña para tomas:
ramnt@ramnt.local's password:
Linux RAMnt 6.12.75+rpt-rpi-2712 #1 SMP PREEMPT Debian 1:6.12.75-1+rpt1 (2026-03-11) aarch64

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
Last login: Sun Jun  7 20:27:25 2026 from 192.168.1.38
ramnt@RAMnt:~ $
```

1.2: Conexión SSH a la Raspberry Pi 5

La opción `-o PubkeyAuthentication=no` se utilizó para forzar autenticación por contraseña y evitar conflictos con claves privadas bloqueadas en el agente SSH de la notebook.

## 1.7 Carga y verificación del módulo

Con el archivo `.ko` disponible en la Raspberry Pi, se cargó el driver mediante `insmod`:

```
1  cd /home/ramnt/Desktop
2  sudo insmod dual_gpio_input_module.ko
```

Luego se verificó que el módulo hubiera quedado registrado entre los módulos activos:

La captura muestra que `dual_gpio_input_module` fue cargado correctamente y quedó residente en el kernel.

```
ramnt@RAMnt:~/Desktop $ sudo insmod dual_gpio_input_module.ko
[sudo] password for ramnt:
ramnt@RAMnt:~/Desktop $ lsmod | grep dual_
dual_gpio_input_module    49152  0
```

### 1.3: Verificación del módulo cargado mediante lsmod

Además de la consulta con `lsmod`, se inspeccionaron los mensajes del kernel mediante `dmesg`. La salida evidencia dos aspectos importantes: primero, el módulo informa que fue cargado sobre el chip `pinctrl-rp1` y que expone los estados en `/dev/tp5_gpio`; segundo, cada lectura posterior registra los valores obtenidos para `gpio_a` y `gpio_b`.

```
1  sudo dmesg | tail
2  cat /dev/tp5_gpio
```

```
[ 215.540795] Modulo cargado: chip=pinctrl-rp1 gpio_a=17 value=0, gpio_b=27 value=0
[ 215.540803] Leer estados en /dev/tp5_gpio
[ 407.907452] Leidos valores: gpio_a=1, gpio_b=1
[ 407.907481] Leidos valores: gpio_a=1, gpio_b=1
[ 411.864760] Leidos valores: gpio_a=1, gpio_b=1
[ 411.864786] Leidos valores: gpio_a=1, gpio_b=1
[ 413.576634] Leidos valores: gpio_a=1, gpio_b=0
[ 413.576662] Leidos valores: gpio_a=1, gpio_b=0
[ 415.091713] Leidos valores: gpio_a=0, gpio_b=1
```

### 1.4: Mensajes del kernel emitidos por el módulo durante la lectura de GPIO

Estos mensajes confirman que el driver no solo se cargó, sino que también ejecutó su operación de lectura y obtuvo cambios coherentes con una secuencia binaria externa.

## 1.8 Lector C de espacio de usuario

Una vez disponible `/dev/tp5_gpio`, se implementó y utilizó el lector ubicado en `tools/gpio_jsonl_reader`. Este programa está escrito en C y se ejecuta en espacio de usuario dentro de la Raspberry Pi. Su función es abrir el dispositivo de carácter, leer el texto devuelto por el módulo, normalizar los valores y emitir una muestra por línea en formato JSON Lines.

La compilación del lector también se realizó previamente en la notebook, ya que el binario final debía ejecutarse en ARM64:

```
1  cd tools/gpio_jsonl_reader
2  make CC=aarch64-linux-gnu-gcc TARGET=build/gpio_jsonl_reader-aarch64
```

Luego se transfirió el binario a la Raspberry Pi y se le otorgaron permisos de ejecución:

```
1  scp -o PubkeyAuthentication=no build/gpio_jsonl_reader-aarch64 ramnt@ramnt.
2  ssh -o PubkeyAuthentication=no ramnt@ramnt.local 'chmod +x /home/ramnt/Desk
```

La ejecución recomendada es:

```
ssh -o PubkeyAuthentication=no ramnt@ramnt.local '/home/ramnt/Desktop/gpio_json
```

En cada ciclo, el programa abre `/dev/tp5_gpio`, lee la muestra completa, cierra el dispositivo, espera 500 ms y repite. La salida producida tiene la forma:

```
{"timestamp_ms":1710000000000,"seq":1,"source":"device","value_a":0,"value_b":1
```

Los campos `gpio_a_value` y `gpio_b_value` representan los bits leídos. A partir de ellos, el lector calcula `binary_code`, `binary_value` y `normalized_value`. Este último se utiliza para graficar el valor sobre un eje normalizado entre 0 y 1.

## 1.9 Envío de muestras hacia la notebook

El programa C escribe cada muestra por `stdout`. Como el proceso se ejecuta remotamente por SSH, esa salida viaja de la Raspberry Pi hacia la notebook. En la notebook se encadena el stream con `curl`, enviando cada línea JSON al endpoint HTTP del dashboard:

```
1  ssh -o PubkeyAuthentication=no ramnt@ramnt.local '/home/ramnt/Desktop/gpio_
2  | while IFS= read -r line; do
3      printf '%s\n' "$line" \
4      | curl -fsS -X POST -H 'Content-Type: text/plain' --data-binary @-
5  done
```

## 1.10 Dashboard web en Docker

La visualización se encuentra en `tools/web_content`. La aplicación se ejecuta en Docker y expone una página local con tarjetas de estado, último valor de cada GPIO, cantidad de muestras recibidas, código binario actual y gráfico temporal.

La configuración utilizada corresponde al modo manual, en el cual el contenedor no ejecuta SSH por sí mismo, sino que espera recibir muestras mediante `POST /api/samples`:

```
1  cd tools/web_content
2  cp .env.example .env
```

Configuración relevante:

```
1  WEB_PORT=8080
2  MAX_SAMPLES=10
3  REMOTE_COMMAND=
```

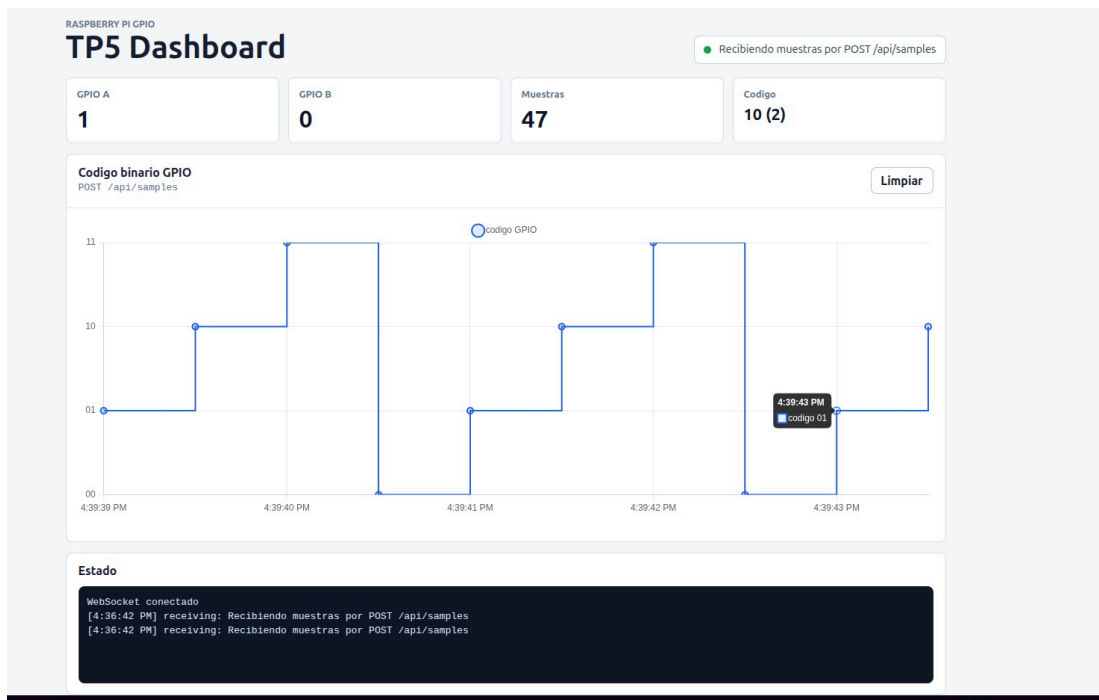
Luego se levanta el servicio:

```
docker compose up --build
```

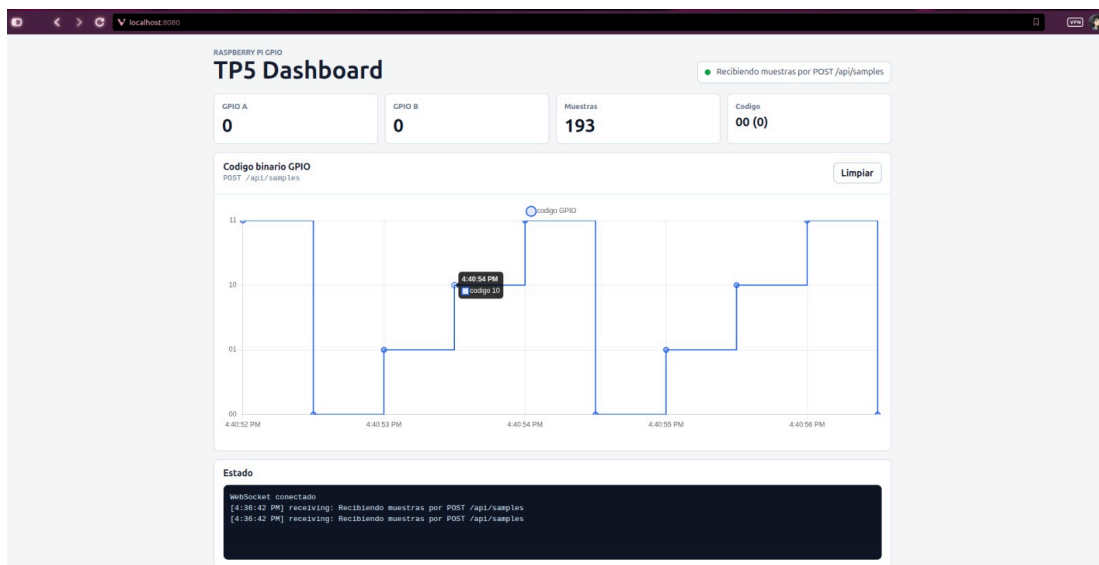
La interfaz queda disponible en:

```
http://localhost:8080
```

En las capturas del dashboard se observa el estado `Recibiendo muestras por POST /api/samples`, la cantidad acumulada de muestras y el gráfico escalonado del código GPIO. La forma escalonada es consistente con la secuencia generada por Arduino y con la conversión 00, 01, 10, 11 realizada por el lector.



1.5: Dashboard recibiendo muestras y graficando el código GPIO



1.6: Evolución temporal de la secuencia binaria en la página web

## 2 Análisis técnico

### 2.1 Separación entre kernel space y user space

La solución respeta una separación adecuada entre el espacio del kernel y el espacio de usuario. El módulo cargable concentra las operaciones que requieren privilegios y conocimiento del hardware: reserva de GPIO, lectura de valores y registro del dispositivo de carácter. El lector C, en cambio, opera con primitivas estándar de espacio de usuario: abre un archivo, lee texto, lo interpreta y produce JSON.

Esta separación es importante porque evita exponer punteros virtuales del kernel o direcciones internas como contrato con la aplicación. En su lugar, se utiliza una interfaz convencional de Linux: un nodo en `/dev`. Este enfoque es más estable, más fácil de probar y menos riesgoso desde el punto de vista de seguridad y mantenimiento.

### 2.2 Coherencia de los datos observados

El circuito genera cuatro estados posibles y el dashboard representa esos estados como códigos binarios. El lector toma los valores individuales, los normaliza a bits y calcula:

```
1  binary_value = (gpio_a_value << 1) | gpio_b_value
2  normalized_value = binary_value / 3
```

Por este motivo, la página puede mostrar tanto el código binario (00, 01, 10, 11) como un nivel numérico normalizado para graficación. La evidencia visual muestra transiciones escalonadas, lo cual coincide con el comportamiento esperado para una secuencia digital discreta con una permanencia temporal fija.

### 2.3 Consideraciones de hardware

La Raspberry Pi 5 utiliza GPIO de 3.3 V. Por lo tanto, cualquier señal externa proveniente de una placa de 5 V debe adaptarse antes de llegar a los pines de la Raspberry Pi. Este

La práctica, por tanto, no se limita al software. El correcto funcionamiento del driver y del dashboard depende de que las señales eléctricas lleguen a la Raspberry Pi con niveles lógicos válidos y seguros.

# 3 Conclusión

El trabajo práctico consistió en construir una cadena completa de adquisición, procesamiento y visualización de señales digitales utilizando una Raspberry Pi 5 como sistema embebido Linux. Para generar una entrada controlada, se programó un Arduino con una secuencia binaria periódica de dos bits (00, 01, 10, 11). Esas señales se conectaron al circuito de entrada de la Raspberry Pi, contemplando que sus GPIO trabajan a 3.3 V y requieren niveles eléctricos seguros.

La etapa principal del TP fue la construcción de un módulo de kernel para Raspberry Pi 5 mediante compilación cruzada. El módulo no se compiló en la placa, sino en la notebook, configurando el entorno con `ARCH=arm64`, `CROSS_COMPILE=aarch64-linux-gnu-` y un árbol de kernel preparado para la versión ejecutada por la Raspberry Pi. Esta etapa permitió generar el archivo `dual_gpio_input_module.ko` para arquitectura ARM64 sin utilizar la Raspberry Pi como máquina de compilación. Antes de transferirlo, el artefacto podía verificarse con `file` para confirmar que correspondiera a AArch64 y no a la arquitectura nativa de la notebook.

Una vez generado el módulo, se lo copió por SSH a la Raspberry Pi y se cargó en el kernel mediante `insmod`. El driver registró el dispositivo de carácter `/dev/tp5_gpio`, reservó dos GPIO de entrada y expuso sus valores en formato textual cada vez que el dispositivo era leído. La correcta inserción se verificó con `lsmod`, mientras que los mensajes de `dmesg` permitieron comprobar tanto la carga del módulo como las lecturas sucesivas de las señales.

Sobre esa interfaz del kernel se ejecutó un programa C de espacio de usuario, también compilado previamente en la notebook para ARM64. Dicho programa abrió `/dev/tp5_gpio`, leyó los valores entregados por el driver, los normalizó como bits, calculó el código binario correspondiente y emitió una muestra por línea en formato JSON Lines. Al ejecutarse remotamente por SSH, la salida estándar del programa viajó desde la Raspberry Pi hacia la notebook.

Finalmente, la notebook levantó una aplicación web dockerizada que expuso el dashboard en `http://localhost:8080`. Cada línea JSON recibida desde la Raspberry Pi fue enviada al endpoint `POST /api/samples`, permitiendo representar en tiempo real el estado de los GPIO, el código binario actual, la cantidad de muestras recibidas y una gráfica temporal escalonada. De esta forma, el TP integró generación de señales, driver de kernel, compilación cruzada, ejecución remota por SSH, comunicación entre espacio de kernel y espacio de usuario, serialización de datos y visualización web en un único flujo funcional.



En síntesis, se comprobó que un módulo de kernel puede desarrollarse y compilarse fuera del dispositivo de destino, transferirse a una Raspberry Pi, cargarse dinámicamente en el kernel y utilizarse como puente controlado entre hardware y aplicaciones de usuario. La solución final dejó una frontera clara: el kernel se encargó de leer el hardware y exponer `/dev/tp5_gpio`, mientras que el espacio de usuario se ocupó de transformar, transportar y visualizar la información.